DOKUMENTACJA PROJEKTOWA

Komunikator internetowy

Voice messenger

1. Wstęp

1.1. Cel Projektu

Głównym celem niniejszej pracy było zaprojektowanie i zaimplementowanie skalowalnego systemu komunikacji w czasie rzeczywistym, nazwanego roboczo "Voice Messenger". System ten łączy w sobie cechy klasycznych komunikatorów tekstowych (czat, historia, statusy) z możliwościami transmisji głosu i wideo.

Kluczowe cele szczegółowe to:

- **Wydajność i Skalowalność:** Opracowanie architektury zdolnej do obsługi rosnącej liczby użytkowników oraz dynamicznego skalowania zasobów w zależności od obciążenia.
- **Optymalizacja Transmisji Mediów:** Implementacja **hybrydowego mechanizmu komunikacji**, który używa jednej z dwóch metod przesyłu danych (bezpośrednią lub z wykorzystaniem serwera pośredniczącego) w zależności od tego, czy rozmowa jest prywatna, czy grupowa. Pozwala to na drastyczną redukcję zużycia pasma sieciowego po stronie klienta oraz zmniejsza wykorzystanie serwerów.
- **Niezawodność (Reliability):** Zapewnienie ciągłości działania usług kluczowych, tak aby awaria jednego z modułów (np. analityki) nie wpływała na możliwość prowadzenia rozmów.

Projekt realizowany jest z naciskiem na nowoczesne standardy bezpieczeństwa (OIDC/OAuth2), modułowość (mikroserwisy) oraz automatyzację procesu wdrażania (Docker).

1.2. Geneza i Motywacja

Współczesne systemy komunikacyjne muszą sprostać rosnącym wymaganiom użytkowników dotyczącym jakości połączeń i dostępności usług. Wybór technologii i architektury podyktowany był następującymi czynnikami:

Protokół **WebRTC** (Web Real-Time Communication) stał się standardem przemysłowym dla komunikacji w przeglądarkach. Umożliwia on bezpośrednią wymianę danych (głos, wideo) między klientami, eliminując potrzebę instalowania zewnętrznych wtyczek.

- **Problematyka NAT i Firewalli**: Bezpośrednie połączenia P2P są często blokowane przez routery NAT i Firewalle korporacyjne. Projekt adresuje to wyzwanie poprzez integrację serwera **Coturn** (implementacja protokołów STUN/TURN), który pozwala na zestawienie połączenia nawet w restrykcyjnych środowiskach sieciowych.
- **Ograniczenia modelu Mesh**: Czysty model P2P (Full Mesh) przestaje być wydajny przy większej liczbie uczestników (złożoność rośnie kwadratowo).

Dlatego w projekcie zdecydowano się na podejście hybrydowe z wykorzystaniem SFU.

Fundamentem platformy jest **architektura rozproszona**, która gwarantuje elastyczność i wydajność niezbędną dla systemów czasu rzeczywistego. Dekompozycja systemu na niezależne, luźno powiązane komponenty zapewnia kluczowe korzyści operacyjne:

- **Niezależne skalowanie**: Możliwość uruchomienia wielu instancji chat-service przy jednej instancji user-service w godzinach szczytu.
- **Separację odpowiedzialności**: Awaria modułu AI (analiza toksyczności) nie wpływa na możliwość prowadzenia rozmów głosowych, co zwiększa ogólną dostępność systemu.
- **Poliglotyzm technologiczny**: Możliwość wykorzystania Javy (Spring Boot) do logiki biznesowej oraz Go (LiveKit) do wysokowydajnego strumieniowania mediów.

1.3. Zakres Pracy

Warstwa Backendowa (Server-Side Logic & Orchestration)

Projekt i implementacja kompletnego, rozproszonego systemu opartego na ekosystemie Java 21 oraz Spring Cloud. Prace obejmowały:

- **Projekt Architektury Zdarzeniowej (Event-Driven)**: Implementacja asynchronicznej komunikacji między serwisami przy użyciu brokera RabbitMQ. Obejmuje to dystrybucję wiadomości czatu lub powiadomienia o statusach znajomych.
- **Orkiestracja Ruchu i Bezpieczeństwo**: Konfiguracja Spring Cloud Gateway jako centralnego punktu wejścia, obsługującego routing dynamiczny, terminację SSL oraz propagację kontekstu bezpieczeństwa (Token Relay) do mikroserwisów ukrytych w sieci prywatnej.
- **Obsługa Czasu Rzeczywistego (Real-Time)**: Implementacja dedykowanych rozwiązań dla protokołów WebSocket (STOMP) oraz WebRTC Signaling. Opracowanie niestandardowych mechanizmów uwierzytelniania sesji WebSocket (Custom Handshake Interceptors), które wykraczają poza standardowe ramy protokołu HTTP.
- **Zarządzanie Stanem i Persystencja**: Projekt schematów bazodanowych dla MongoDB (dokumentowy model wiadomości i struktur serwerów) oraz integracja z MinIO (S3) dla składowania obiektów binarnych.

Warstwa Frontendowa (React, TypeScript)

Stworzenie responsywnej aplikacji typu SPA (Single Page Application) z naciskiem na User Experience (UX). Wykorzystanie:

- **State Management**: Zarządzanie złożonym stanem aplikacji (lista serwerów, aktywny kanał, statusy znajomych) przy użyciu Context API i Reducerów.
- **Custom Hooks**: Hermetyzacja logiki biznesowej, np. useWebRTCCall do obsługi maszyny stanów połączenia głosowego.
- **Form Validation**: Walidacja formularzy (logowanie, rejestracja) przy użyciu biblioteki Zod.

Infrastruktura i DevOps Przygotowanie środowiska uruchomieniowego w oparciu o konteneryzację **Docker**. Konfiguracja obejmuje:

- **Identity Provider**: Serwer Keycloak do zarządzania tożsamością i rolami użytkowników.
- **Object Storage**: Serwer MinIO do przechowywania plików statycznych (awatary).
- **Transversal**: Serwer Coturn (STUN/TURN) dla obsługi NAT.
- **Telemetry**: Monitoring parametrów sieciowych (Packet Loss, RTT, Jitter) zbieranych przez Analytics Service w celu diagnostyki jakości połączeń.

2. Wymagania Funkcjonalne

System realizuje szereg kluczowych funkcji umożliwiających kompleksową komunikację i zarządzanie społecznością. Poniżej przedstawiono szczegółowy opis poszczególnych modułów funkcjonalnych.

2.1. System Użytkowników i Uwierzytelniania

Fundamentem bezpieczeństwa i personalizacji platformy jest zaawansowany system zarządzania tożsamością.

- **Rejestracja i Logowanie:** Proces uwierzytelniania jest zdelegowany do zewnętrznego dostawcy tożsamości (Identity Provider) – **Keycloak**. Obsługuje on standardy **OAuth2** oraz **OpenID Connect (OIDC)**, co gwarantuje najwyższy poziom bezpieczeństwa. Aplikacja kliencka nie przechowuje haseł, a jedynie tokeny dostępu (Access Token) i odświeżania (Refresh Token) w formacie **JWT (JSON Web Token)**.
- **Zarządzanie Profilem:** Użytkownicy mogą zarządzać swoimi danymi oraz ustawiać zdjęcia profilowe (przechowywane w systemie **MinIO**)
- **System Znajomych:** Platforma umożliwia wysyłanie zaproszeń do znajomych, ich akceptację oraz usuwanie niechcianych użytkowników. Relacje te są podstawą do inicjowania prywatnych konwersacji (DM).

2.2. Struktura Serwerów i Kanałów

Organizacja komunikacji grupowej opiera się na hierarchicznej strukturze:

- **Serwery (pokoje):** Użytkownicy mogą tworzyć własne serwery lub dołączać do istniejących. Każdy serwer stanowi odrębną przestrzeń administracyjną z własną listą członków i uprawnieniami.
- **Kanały:** W ramach serwera komunikacja podzielona jest na kanały. Wyróżniamy dwa podstawowe typy kanałów:
    - **Kanały Tekstowe:** Służą do asynchronicznej i synchronicznej wymiany wiadomości oraz linków. Historia wiadomości jest persystentna.
    - **Kanały Głosowe:** Umożliwiają natychmiastowe dołączenie do rozmowy grupowej (audio/wideo). Użytkownik wchodząc na kanał automatycznie łączy się ze strumieniem mediów innych uczestników.

2.3. System Czat (Wiadomości Tekstowe)

Moduł czatu zapewnia wymianę informacji w czasie rzeczywistym z minimalnym opóźnieniem.

- **Real-time Messaging:** Dzięki wykorzystaniu protokołu **STOMP over WebSockets** oraz brokera wiadomości **RabbitMQ**, wiadomości trafiają do odbiorców niemal natychmiast po wysłaniu.
- **Historia Wiadomości:** Wszystkie wiadomości są archiwizowane w bazie danych **MongoDB**, co pozwala na przeglądanie historii rozmów po ponownym zalogowaniu lub przewinięciu okna czatu.
- **Powiadomienia:** System śledzi wiadomości i powiadamia odbiorcę o nowych wiadomościach.
- **Bezpośrednie Wiadomości (DM):** Poza kanałami publicznymi, użytkownicy mogą prowadzić prywatne konwersacje 1:1.
- **Wizualizacja Nieprzeczytanych Wiadomości (Unread Counters):** System implementuje mechanizm śledzenia aktywności, prezentując liczbowy wskaźnik nieprzeczytanych wiadomości. Licznik ten jest widoczny w interfejsie użytkownika w dwóch kluczowych miejscach:
    - Przy nazwach kanałów tekstowych na serwerach.
    - W sekcji Wiadomości Bezpośrednich (DM) przy awatarze znajomego. pozwala użytkownikowi na natychmiastową identyfikację aktywnych wątków.

2.4. Komunikacja Audio/Wideo (WebRTC)

Najbardziej złożony element systemu, odpowiadający za przesyłanie strumieni multimedialnych. System wykorzystuje **Architekturę Hybrydową**.

- **Rozmowy 1-na-1 (P2P):** Realizowane bezpośrednio między użytkownikami (Peer-to-Peer) przy użyciu własnego serwisu sygnalizacyjnego i serwera **Coturn (TURN)**. Zapewnia to prywatność i odciąża serwer od przetwarzania mediów.
- **Architektura SFU (Selective Forwarding Unit):** Wykorzystanie serwera **LiveKit** jako SFU pozwala na efektywne zarządzanie pasmem. Zamiast łączyć każdego z każdym (Mesh), co obciąża łącza użytkowników, każdy klient wysyła strumień do serwera, który następnie dystrybuuje go do pozostałych uczestników.
- **Adaptacyjna Jakość:** System dynamicznie dostosowuje jakość wideo (Simulcast) do warunków sieciowych odbiorcy, zapewniając płynność rozmowy nawet przy słabszym łączu.
- **Udostępnianie Ekranu:** Możliwość przesyłania obrazu pulpitu lub konkretnego okna aplikacji do innych uczestników kanału.
- **Sygnalizacja:** Własny serwis sygnalizacyjny (signaling-service) zarządza nawiązywaniem połączeń, wymianą ofert/odpowiedzi SDP (Session Description Protocol) oraz kandydatów ICE.

2.5. Automatyczna Moderacja Treści (AI Integration)

W celu zapewnienia bezpieczeństwa użytkowników i higieny dyskusji, zaimplementowano dedykowany mikroserwis ai-service.

- **Analiza Językowa**: Moduł ten integruje się z modelem językowym, umożliwiając wykrywanie treści toksycznych (wulgaryzmy, mowa nienawiści).
- **Przetwarzanie Asynchroniczne**: Proces moderacji odbywa się w tle, nie wpływając na szybkość dostarczania wiadomości. W przypadku wykrycia naruszenia regulaminu, system automatycznie flaguje wiadomość i ukrywa jej treść przed innymi użytkownikami.

2.6. Telemetria i Analityka Sieciowa (Analytics)

System wyposażono w zaawansowany moduł analityczny analytics-service, odpowiedzialny za gromadzenie i przetwarzanie metryk technicznych oraz biznesowych.

- **Monitoring QoS (Quality of Service):** Kluczową funkcją jest śledzenie parametrów jakości połączeń WebRTC, takich jak opóźnienia (RTT – Round Trip Time), utrata pakietów (Packet Loss) oraz zmienność opóźnienia (Jitter). Pozwala to na bieżącą diagnostykę problemów sieciowych użytkowników.

- **Metryki Aktywności:** Serwis agreguje dane dotyczące obciążenia serwerów (liczba aktywnych pokoi, zużycie pasma) oraz zaangażowania użytkowników, co umożliwia administratorom podejmowanie decyzji o skalowaniu infrastruktury.

3. Wymagania Niefunkcjonalne

Wymagania niefunkcjonalne określają atrybuty jakościowe systemu, kluczowe dla komfortu użytkowania aplikacji typu Real-Time.

3.1. Wydajność i Niskie Opóźnienia (Low Latency)

- Dla komunikacji głosowej i wideo opóźnienia muszą być minimalne (poniżej 200-300ms dla komfortowej rozmowy). Zastosowanie protokołu **UDP** w warstwie transportowej WebRTC eliminuje narzut związany z potwierdzaniem pakietów (TCP), co jest kluczowe dla mediów strumieniowych.
- Wykorzystanie **WebSockets** dla sygnalizacji i czatu zapewnia stałe, dwukierunkowe połączenie, eliminując narzut HTTP Handshake przy każdej wiadomości.

3.2. Skalowalność i Dostępność

- **Architektura Mikroserwisów:** Podział systemu na niezależne komponenty pozwala na niezależne skalowanie każdego z nich. Np. przy wzroście liczby wiadomości można zwiększyć liczbę instancji chat-service bez konieczności skalowania room-service.
- **Bezstanowość (Statelessness):** Serwisy backendowe (z wyjątkiem specyficznych przypadków jak WebSocket session cache) są zaprojektowane jako bezstanowe, co ułatwia ich horyzontalne skalowanie i zarządzanie przez orkiestrator.
- **Broker Wiadomości (RabbitMQ):** Zastosowanie asynchronicznej komunikacji między serwisami zapobiega zatorom i pozwala buforować żądania w momentach szczytowego obciążenia.

3.3. Bezpieczeństwo

- **Szyfrowanie Transportu:** Cała komunikacja HTTP odbywa się po HTTPS (TLS/SSL), a połączenia WebSocket po WSS. Strumienie WebRTC są domyślnie szyfrowane (DTLS/SRTP).
- **Uwierzytelnianie i Autoryzacja:** Zastosowanie standardu branżowego OAuth2/OIDC (Keycloak) z tokenami JWT. Gateway weryfikuje tokeny przed przepuszczeniem żądania do serwisów wewnętrznych.

- **Izolacja Sieciowa:** W środowisku produkcyjnym bazy danych i brokerzy nie są wystawieni bezpośrednio do Internetu, a dostępne jedynie w sieci wewnętrznej klastra.

3.4. Dostępność za NAT-em (NAT Traversal)

- Wykorzystanie protokołu **ICE (Interactive Connectivity Establishment)** pozwala na nawiązywanie połączeń P2P nawet gdy użytkownicy znajdują się za domowymi routerami (NAT).
- Wsparcie dla serwerów **STUN/TURN** gwarantuje zestawienie połączenia nawet w restrykcyjnych sieciach korporacyjnych, gdzie bezpośrednie połączenie P2P jest blokowane.

4. Stos Technologiczny

Dobór technologii podyktowany został wymaganiami nowoczesności i wydajności oraz wsparcia społeczności.

4.1. Backend (Warstwa Serwerowa)

- **Język Programowania:** Java 21
- **Framework:** Spring Boot 3.4.1
- **Architektura Rozproszona:** Spring Cloud 2024.0.0
- **API Gateway:** Spring Cloud Gateway
- **Bezpieczeństwo:** Spring Security 6
- **Komunikacja WebSocket:** Spring WebSocket
- **Klient HTTP:** Spring Cloud OpenFeign

4.2. Frontend (Warstwa Kliencka)

- **Framework UI:** React 19
- **Język:** TypeScript 5.9
- **Build Tool:** Vite
- **WebRTC Client:** LiveKit Client SDK
- **WebSocket Client:** @stomp/stompjs
- **Zarządzanie stanem:** Wykorzystanie Context API oraz dedykowanych hooków (np. useWebRTCCall, useSignaling).

- **Walidacja:** Zod

4.3. Bazy Danych i Przechowywanie Danych

- **MongoDB:** Dedykowane bazy danych dla serwisu czatu (chat-service), pokoi (room-service), użytkowników (user-service) i analityki (analytics-service). Wybrana ze względu na elastyczny schemat (dokumentowa) i łatwą skalowalność, co jest idealne dla przechowywania historii rozmów.
- **PostgreSQL:** Relacyjna baza danych wykorzystywana przez **Keycloak** do przechowywania tożsamości, ról i konfiguracji realmów. Gwarantuje spójność transakcyjną danych autoryzacyjnych.
- **Redis:** Pamięć podręczna (In-memory data structure store) wykorzystywana przez **LiveKit** do zarządzania stanem sesji i pokoi w czasie rzeczywistym.
- **MinIO:** Kompatybilny z S3 system przechowywania obiektów (Object Storage), używany do składowania plików statycznych, takich jak awatary użytkowników czy załączniki w czacie.

4.4. Infrastruktura i Messaging

- **RabbitMQ:** Broker wiadomości implementujący protokół AMQP. Używany do asynchronicznej komunikacji między serwisami (np. zdarzenie "nowa wiadomość" wyzwala powiadomienie push w innym serwisie) oraz jako STOMP Relay dla WebSockets.
- **Docker & Docker Compose:** Konteniazyracja wszystkich komponentów systemu, zapewniająca spójność środowiska developerskiego, testowego i produkcyjnego.
- **Keycloak:** Serwer zarządzania tożsamością i dostępem (IAM). Centralizuje logikę logowania, rejestracji i resetowania haseł.
- **LiveKit Server:** Wysokowydajny serwer WebRTC (SFU) napisany w Go. Odpowiada za miksowanie i dystrybucję strumieni audio/wideo.
- **Coturn:** Serwer implementujący protokoły **STUN/TURN**. Kluczowy element infrastruktury umożliwiający obejście NAT (NAT Traversal) i zestawienie połączeń P2P, gdy bezpośrednia komunikacja jest blokowana przez firewalle.

4.5. Protokoły Komunikacyjne

- **REST:** Służy do operacji typu Request-Response (np. pobieranie listy serwerów, edycja profilu).
- **WebSocket / STOMP:** Używany do dwukierunkowej komunikacji w czasie rzeczywistym (np. czat).

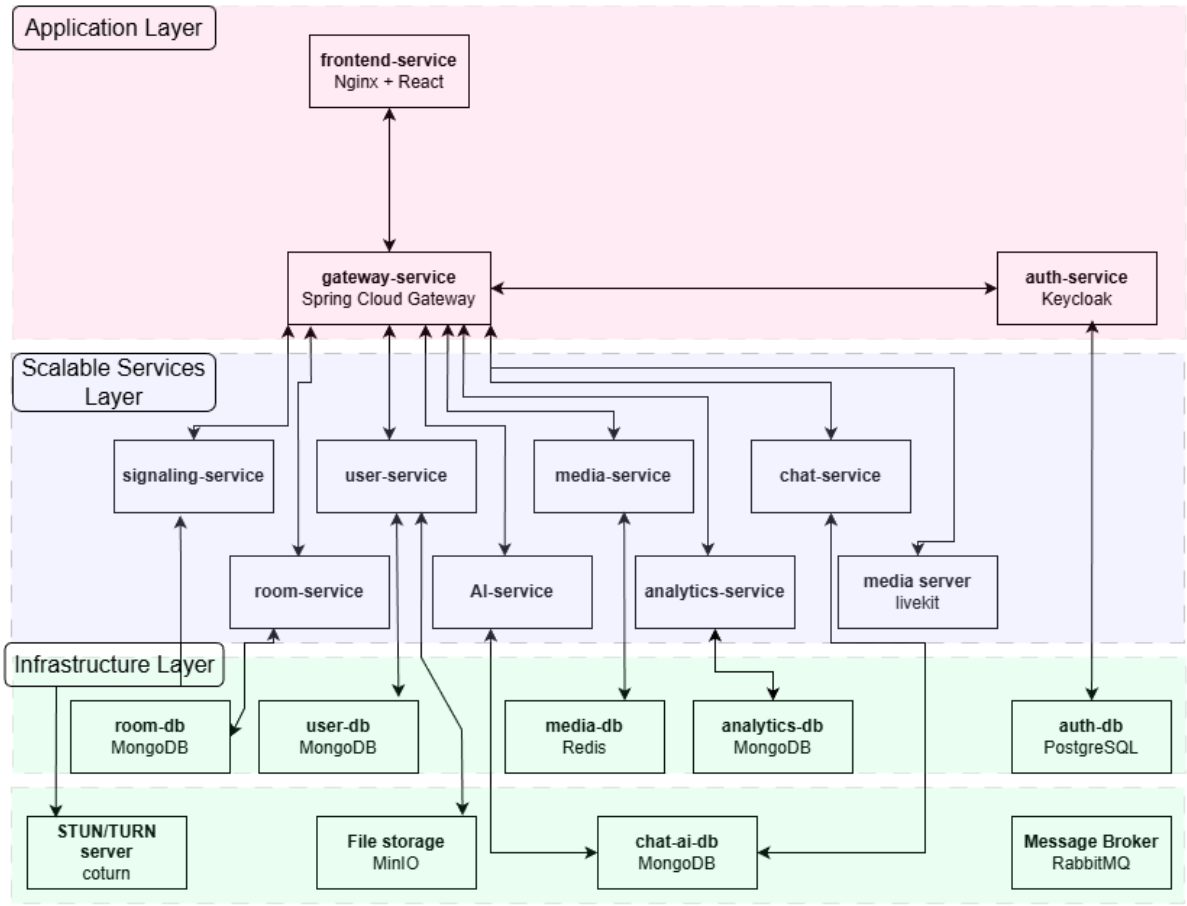
- **WebRTC (ICE/DTLS/SRTP):** Służy do bezpośredniej wymiany mediów (głos, wideo, ekran).
- **AMQP:** Asynchroniczna kolejka wiadomości między mikroserwisami.

5. Architektura Mikroserwisów

System **Voice Messenger** został zaprojektowany w oparciu o nowoczesną architekturę mikroserwisów, co zapewnia wysoką skalowalność, niezależność wdrażania poszczególnych modułów oraz łatwość w utrzymaniu i rozwoju (Maintainability). Całość rozwiązania jest orkiestrowana przez konteneryzację Docker, a komunikacja między usługami odbywa się w modelu mieszanym: synchronicznym (**REST API**) oraz asynchronicznym (**Event-Driven**).

Sercem systemu jest **API Gateway**, który zarządza ruchem przychodzącym od klientów (aplikacja webowa) i dystrybuuje go do odpowiednich serwisów backendowych, pełniąc jednocześnie funkcję strażnika bezpieczeństwa.

5.1. Diagram Ogólny Systemu



5.2.2. API Gateway – Centralny Punkt Wejścia

Usługa **gateway-service**, oparta na **Spring Cloud Gateway** i modelu reaktywnym (Netty), pełni rolę jedynego punktu styku (Entry Point) dla klientów zewnętrznych. Realizuje ona kluczowe zadania infrastrukturalne:

1. **Routing (Trasowanie)**: Na podstawie predykatów ścieżki (Path Predicates) ruch jest kierowany do odpowiednich klastrów usług, ukrywając wewnętrzną topologię sieci. Przykładowe mapowania:
    - /ws/signal/** signaling-service (Sygnalizacja WebRTC).
    - /api/chat/** chat-service (Komunikacja tekstowa).
    - /api/media/** media-service (Integracja SFU).
2. **Autoryzacja (Resource Server)**: Gateway działa jako strażnik (Gatekeeper). Weryfikuje podpis cyfrowy tokenów JWT (RS256) przy użyciu kluczy publicznych pobieranych z serwera **Keycloak**. Mechanizm **TokenRelay** przekazuje zweryfikowaną tożsamość (sub, roles) do serwisów podrzędnych.
3. **Sanityzacja Żądań**: Filtr RemoveRequestHeader usuwa niebezpieczne nagłówki przed wpuszczeniem ruchu do sieci wewnętrznej.

5.2.3. Uwierzytelnienie w WebSocketach

Standardowy protokół WebSocket w przeglądarkach (JavaScript API) nie pozwala na łatwe dodawanie niestandardowych nagłówków (takich jak Authorization) do pierwszego żądania HTTP Upgrade (Handshake). Stanowiło to istotne wyzwanie inżynierskie w projekcie.

Zastosowane rozwiązanie:

- **Gateway:** Skonfigurowano reguły bezpieczeństwa tak, aby przepuszczać ruch na ścieżkach /ws/** bez wstępnej weryfikacji JWT na poziomie Gatewaya (permitAll()). Weryfikacja została przesunięta "w głąb" systemu.
- **Serwisy Aplikacyjne (Signaling/Chat):** Uwierzytelnianie zaimplementowano w warstwie protokołu **STOMP**. W klasie SignalMessagingConfig.java wdrożono niestandardowy ChannelInterceptor.
    - **Logika Interceptora:** Przechwytuje on ramkę CONNECT protokołu STOMP. W tym momencie klient (frontend) przesyła token JWT jako atrybut ramki (nativeHeaders). Interceptor wyciąga token, dekoduje go i waliduje tożsamość użytkownika przed zestawieniem sesji.

- **Obsługa Query Params:** Dodatkowo, w konfiguracji bezpieczeństwa serwisów włączono setAllowUriQueryParameter(true), co umożliwia przekazanie tokena w adresie URL (?access_token=...) w scenariuszach, gdzie nagłówki STOMP nie są wspierane przez klienta.

5.3. Model Hybrydowy WebRTC

Unikalną cechą systemu Voice Messenger jest zastosowanie **hybrydowego modelu komunikacji głosowej i wideo**, który używa jednej z dwóch architektur. System wykorzystuje **P2P (Peer-to-Peer)** lub **SFU (Selective Forwarding Unit)** w zależności od kontekstu rozmowy. Podejście to optymalizuje koszty infrastruktury oraz wydajność sieciową.

5.3.1. Porównanie Architektur: P2P vs SFU

Cecha	Tryb P2P (Rozmowy Prywatne 1:1)	Tryb SFU (Kanały Głosowe / LiveKit)
Topologia	Mesh (Każdy z każdym)	Gwiazda (Wszyscy do serwera)
Pośrednik	Brak (bezpośrednio) lub Coturn (TURN)	Media Service + LiveKit Server
Obciążenie Serwera	Minimalne (tylko sygnalizacja)	Wysokie (miksowanie i routing mediów)
Opóźnienia	Najniższe możliwe	Nieznacznie wyższe (narzut serwera)

5.3.2. Implementacja

Logika decyzyjna zaimplementowana jest w warstwie frontendu i działa w sposób przezroczysty dla użytkownika.

1. **Scenariusz P2P:** Gdy użytkownik inicjuje połączenie prywatne, aplikacja wykorzystuje natywne API WebRTC przeglądarki.
    - **Broker Sygnalizacyjny:** signaling-service pośredniczy w wymianie ofert SDP (Offer/Answer) oraz kandydatów ICE poprzez WebSocket. Serwer nie przetwarza strumieni mediów, a jedynie metadane połączenia.
    - **NAT Traversal:** Wykorzystywany jest własny serwer STUN/TURN (kontener coturn), aby zestawić połączenie bezpośrednie między użytkownikami znajdującymi się za NAT-em.
2. **Scenariusz SFU (Kanały Głosowe Serwera):** Gdy użytkownik dołącza do kanału głosowego (selectedChannel.type === 'VOICE'), mechanizm P2P jest pomijany.
    - Aplikacja wysyła żądanie do media-service o token dostępu do pokoju.

- Backend generuje token JWT podpisany kluczem API LiveKit i zwraca go do klienta.
- Klient łączy się z instancją **LiveKit Server** (SFU). W tym modelu każdy użytkownik przesyła tylko jeden strumień (uplink) do serwera, a pobiera N-1 strumieni od innych (downlink). Drastycznie oszczędza to pasmo uploadu użytkownika (wymagane pasmo jest stałe, niezależnie od liczby uczestników), co jest kluczowe dla rozmów grupowych.

5.4. Infrastruktura sieciowa i problem NAT

Wdrożenie systemów czasu rzeczywistego (RTC) w środowisku rozproszonym wiąże się z problemem translacji adresów sieciowych (NAT) oraz firewalli, które blokują bezpośrednie połączenia P2P. System Voice Messenger adresuje te wyzwania poprzez zaawansowaną konfigurację infrastruktury.

5.4.1. Serwer Coturn (STUN/TURN)

W infrastrukturze Dockerowej (docker-compose.yml) uruchomiono dedykowaną instancję serwera **Coturn**, realizującą dwa protokoły:

1. **STUN (Session Traversal Utilities for NAT):** Pozwala klientom odkryć swój publiczny adres IP i port (tzw. adres *Server Reflexive*). Jest to wystarczające dla większości typów NAT (tzw. Cone NAT).
2. **TURN (Traversal Using Relays around NAT):** W przypadku restrykcyjnych, symetrycznych NAT-ów (często spotykanych w sieciach korporacyjnych lub mobilnych LTE), bezpośrednie połączenie P2P jest niemożliwe. Coturn działa wtedy jako **przekaźnik (Relay)** – odbiera pakiety medialne od jednego klienta na publicznym porcie i przesyła je do drugiego.

Implementacja w useWebRTCCall.ts dynamicznie pobiera listę serwerów ICE z backendu, co pozwala na centralne zarządzanie konfiguracją TURN bez konieczności aktualizacji kodu klienta.

6. Komponenty Systemu

System Voice Messenger składa się z zestawu wyspecjalizowanych mikroserwisów, z których każdy realizuje ściśle określony wycinek domeny biznesowej. Poniżej przedstawiono szczegółową specyfikację implementacyjną poszczególnych usług.

Poniższa tabela przedstawia zestawienie usług uruchamianych w ramach orkiestracji docker-compose. Ze względów bezpieczeństwa, w środowisku produkcyjnym porty wewnętrznych mikroserwisów nie są mapowane na interfejsy publiczne hosta, a dostęp do nich możliwy jest wyłącznie przez API Gateway lub wewnątrz sieci prywatnej.

Komponenty skalowalne:

Kontener	Port (Wewnątrz)	Mapowanie Host	Technologie	Rola w Systemie
frontend-service	80 (TCP)	3000	Nginx, React, Vite	Serwuje pliki statyczne aplikacji SPA (Reverse Proxy).
gateway-service	8080 (TCP)	8080	Spring Cloud Gateway, Netty	Brama API, autoryzacja, routing żądań.
keycloak	9090 (TCP)	9090	Java, Quarkus	Dostawca tożsamości (IdP), zarządzanie tokenami.
chat-service	8082 (TCP)	-	Spring Boot, RabbitMQ	Logika czatu, persystencja wiadomości, WebSocket.
room-service	8086 (TCP)	-	Spring Boot, MongoDB	Zarządzanie strukturą serwerów i uprawnieniami RBAC.
user-service	8087 (TCP)	-	Spring Boot, MinIO	Zarządzanie profilami, znajomymi i awatarami.
signaling-service	8083 (TCP)	-	Spring WebSocket	Sygnalizacja WebRTC (wymiana SDP/ICE).
media-service	8888 (TCP)	-	Spring Boot, JWT	Provider tokenów dostępowych dla LiveKit (SFU).
ai-service	8084 (TCP)	-	Spring Boot, REST	Worker asynchroniczny do moderacji treści (LLM).
analytics-service	8085 (TCP)	-	Spring Boot, MongoDB	Gromadzenie i agregacja metryk sieciowych (QoS).

Komponenty infrastrukturalne (osobne urządzenie):

Kontener	Port (Wewn.)	Mapowanie Host	Obraz Docker	Rola w Systemie
postgres-auth	5432 (TCP)	8542	postgres:16	Relacyjna baza danych dla serwera Keycloak.
mongo-db: chat-db analytics-db room-db user-db	27017 (TCP)	8543-8547	mongo:6.0	Nierelacyjna baza danych dla serwisów Chat, Room i Analytics.
redis-cache	6379 (TCP)	8545	redis:7	Pamięć podręczna (In-Memory) dla sesji LiveKit.
rabbitmq	5672 (TCP), 15672 (TCP)	15672 (Mgmt)	rabbitmq:3-mgmt	Broker wiadomości (AMQP) oraz Relay dla WebSocket (STOMP).
minio	9000 (TCP), 9001 (TCP)	9000, 9001	minio/minio	Magazyn obiektowy S3 (Awatary, Pliki).
livekit-server	7880 (TCP), 7881 (TCP), 50000-50050 (UDP)	7880, 50000-50050	livekit/livekit-server	Serwer mediów (SFU) do miksowania audio/wideo.
coturn	3478 (TCP/UDP), 49160-49200 (UDP)	3478, 49160-49200	coturn/coturn	Serwer STUN/TURN (Relay) dla WebRTC.
frps	7000 (TCP)	7000, 80, 443	snowdreamtech/frps	Serwer tunelujący (uruchomiony na VPS).

6.1.1. Gateway Service (Brama API)

Stos technologiczny: Spring Boot, Spring Cloud Gateway, Spring Security, Netty.

Usługa ta stanowi jedyny publiczny punkt styku z systemem. Nie zawiera logiki biznesowej, lecz pełni kluczowe funkcje infrastrukturalne:

- **Routing Dynamiczny:** Kieruje żądania HTTP oraz połączenia WebSocket do odpowiednich klastrów wewnętrznych na podstawie prefiksów ścieżek (np. /api/chat chat-service).
- **Translacja Tokenów (Token Relay):** Działa jako klient OAuth2. Po pomyślnej weryfikacji podpisu tokena JWT na wejściu, przekazuje go w nagłówku do serwisów podrzędnych, zapewniając propagację kontekstu bezpieczeństwa.
- **Sanityzacja Żądań:** Wykorzystuje filtr RemoveRequestHeader do usuwania nagłówków mogących zdradzać szczegóły infrastruktury lub powodować konflikty wewnątrz sieci.

6.1.2. User Service (Zarządzanie Tożsamością i Relacjami)

Stos technologiczny: Spring Boot, MongoDB, MinIO.

Serwis rozszerza podstawowe dane tożsamościowe (przechowywane w Keycloak) o atrybuty społecznościowe.

- **Synchronizacja Tożsamości:** Zaimplementowano mechanizm "Lazy Sync". Podczas pierwszego logowania serwis przechwytuje token użytkownika i tworzy jego profil w lokalnej bazie danych, co gwarantuje spójność między IdP (Keycloak) a aplikacją.
- **Maszyna Stanów Znajomości:** Relacje między użytkownikami modelowane są jako dokumenty z polami stanu: PENDING (oczekujące), FRIENDS (znajomi). Logika biznesowa blokuje próby wysłania zaproszenia do samego siebie lub duplikowania istniejących relacji.
- **Obsługa Zdjęć profilowych:** Integruje się z systemem MinIO w celu przesyłania plików graficznych. Serwis waliduje format pliku, generuje unikalną nazwę (UUID) i zwraca publiczny adres URL zasobu.

6.1.3. Room Service (Zarządzanie Strukturą Serwerów)

Stos technologiczny: Spring Boot, Spring Data MongoDB.

Odpowiada za hierarchiczną strukturę społeczności (Serwery – Kanały - Użytkownicy).

- **Kontrola Dostępu:** Zaimplementowano logikę weryfikacji uprawnień na poziomie aplikacji. Operacje destrukcyjne (np. usunięcie kanału, banowanie użytkownika) są dozwolone wyłącznie dla użytkownika posiadającego rolę OWNER w danym serwerze.
- **Spójność Danych:** W przypadku usunięcia serwera, serwis publikuje zdarzenie domenowe ServerDeletedEvent na szynę RabbitMQ. Dzięki temu inne serwisy mogą kaskadowo usunąć powiązane dane (np. historię czatu), zachowując integralność systemu.

6.1.4. Chat Service (Komunikacja Tekstowa)

Stos technologiczny: Spring Boot, RabbitMQ (STOMP), MongoDB.

Jest to serwis o najwyższym obciążeniu (High Throughput), obsługujący przesyłanie wiadomości w czasie rzeczywistym.

- **Model Hybrydowy:** Wiadomości są przesyłane natychmiastowo przez WebSocket (dla niskich opóźnień) oraz asynchronicznie zapisywane w bazie MongoDB (dla trwałości historii).
- **Statusy Odczytu (Read Receipts):** Zoptymalizowano mechanizm śledzenia nieprzeczytanych wiadomości. Zamiast przechowywać status dla każdej wiadomości osobno, system przechowuje jeden znacznik czasu lastReadAt dla pary Użytkownik-Kanał. Liczba nieprzeczytanych wiadomości jest obliczana dynamicznie przez zapytanie do bazy z filtrem czasowym.
- **Obsługa Wulgarnych Treści:** Serwis nasłuchuje na zdarzenia z ai-service. W przypadku oznaczenia wiadomości jako wulgarnej, aktualizuje jej status w bazie (isToxic: true) i wysyła korektę do klientów frontendowych.

6.1.5. Signaling Service (Sygnalizacja P2P)

Stos technologiczny: Spring Boot, Spring WebSocket, SockJS.

Serwis ten pełni rolę brokera informacji dla połączeń WebRTC Peer-to-Peer, nie uczestnicząc w przesyłaniu mediów.

- **Bezpieczny Handshake:** Ze względu na ograniczenia przeglądarek w obsłudze nagłówków WebSocket, zaimplementowano niestandardowy ChannelInterceptor. Przechwytuje on ramkę CONNECT protokołu STOMP i weryfikuje token JWT przekazany w parametrach połączenia.
- **Wymiana SDP:** Umożliwia bezpieczne i prywatne przekazywanie ofert (Offer) i odpowiedzi (Answer) sesji WebRTC pomiędzy konkretnymi użytkownikami (Direct Messaging).
- **Negocjacja ICE:** Pośredniczy w wymianie kandydatów sieciowych (adresów IP/portów), niezbędnych do zestawienia bezpośredniego tunelu UDP między klientami.

6.1.6. Media Service (Integracja SFU / LiveKit)

Stos technologiczny: LiveKit Server SDK, JWT.

Działa jako fasada (Facade) zabezpieczająca dostęp do serwera mediów LiveKit.

- **Generowanie Tokenów Dostępowych:** Klient nie łączy się z LiveKit "na dziko". Najpierw musi poprosić Media Service o dostęp. Serwis weryfikuje, czy użytkownik jest członkiem danego kanału głosowego, a następnie generuje krótkożyciowy token JWT podpisany kluczem API (API Secret).
- **Zarządzanie Uprawnieniami:** Token zawiera precyzyjnie zdefiniowane granty, np. canPublish: true (może mówić), canSubscribe: true (może słuchać), co zapobiega nieautoryzowanemu nadawaniu treści.
- **Webhooks:** Serwis odbiera zdarzenia zwrotne od serwera LiveKit (np. ParticipantDisconnected), co pozwala na aktualizację listy aktywnych uczestników na frontendzie w czasie rzeczywistym.

6.1.7. AI Service (Moderacja Treści)

Stos technologiczny: Spring Boot, REST Client.

Działa jako asynchroniczny proces w tle (Background Worker), odciążający główny wątek czatu.

- **Analiza Tekstu:** Serwis nasłuchuje na kolejce RabbitMQ text.analyze. Pobiera treść nowej wiadomości i wysyła ją do zewnętrznego API modelu językowego (LLM).
- **Logika Decyzyjna:** Na podstawie zwróconego wyniku (prawdopodobieństwo wulgaryzmu, mowy nienawiści), serwis podejmuje decyzję o oflagowaniu wiadomości. Jeśli współczynnik pewności przekracza zdefiniowany próg (np. 0.8), publikuje zdarzenie text.result z flagą isToxic.

6.1.8. Analytics Service (Telemetria Sieciowa)

Stos technologiczny: Spring Boot, MongoDB TimeSeries.

Serwis odpowiedzialny za monitorowanie jakości usług (QoS - Quality of Service).

- **Agregacja Metryk:** Odbiera okresowe raporty od klientów frontendowych zawierające statystyki WebRTC (np. Packet Loss, Round Trip Time, Jitter).
- **Optymalizacja Zapisu:** Aby uniknąć przeciążenia bazy danych tysiącami pojedynczych zapytań, zastosowano mechanizm buforowania. Metryki są zbierane w pamięci i zrzucane do bazy danych w paczkach (Batch Insert) co określony interwał czasowy (5 sekund).
- **Struktura Danych:** Wykorzystuje kolekcje typu TimeSeries w MongoDB, co pozwala na wydajne przechowywanie i analizowanie danych szeregów czasowych oraz automatyczne usuwanie starych rekordów (TTL).

6.2.1. Strategia Skalowalności

Architektura wdrożeniowa została podzielona na dwa logiczne stosy w celu optymalizacji zarządzania zasobami:

1. **Warstwa Infrastruktury (Stateful)** – zdefiniowana w infrastructure/docker-compose.yml.
    - Obejmuje komponenty przechowujące stan: PostgreSQL, MongoDB, MinIO oraz RabbitMQ.
    - W podstawowym wdrożeniu działają jako pojedyncze instancje.
    - W środowisku produkcyjnym (np. Kubernetes) są zastępowane przez usługi zarządzane (Managed Services) lub klastry wysokiej dostępności (HA).
2. **Warstwa Aplikacyjna (Stateless)** – zdefiniowana w docker-compose-apps.yml.
    - Obejmuje mikroserwisy Spring Boot.
    - Serwisy te nie przechowują stanu sesji w pamięci operacyjnej (delegują stan do Redis/DB), co pozwala na skalowanie horyzontalne.

6.3. Konfiguracja Zmiennych Środowiskowych

Konfiguracja systemu została wyeksportowana do pliku .env. Umożliwia to łatwą zmianę parametrów bez rekompilacji kodu.

Kategoria	Zmienna	Opis	Przykładowa Wartość
Infrastruktura	INFRA_HOST	Publiczny adres IP hosta lub domena główna.	**3.71.11.3**
	COTURN_EXTERNAL_IP	Adres IP ogłaszany przez serwer TURN (Relay).	**3.71.11.3**
Bezpieczeństwo (Keycloak)	KEYCLOAK_CLIENT_SECRET	Sekret klienta OAuth2 używany przez Gateway.	********
	SPRING_..._ISSUER_URI	Adres wystawcy tokena (Issuer) do walidacji JWT.	**https://auth.voicemessenger...**
	SPRING_..._JWK_SET_URI	Endpoint z kluczami publicznymi do weryfikacji podpisu.	**http://keycloak:9090/.../certs**
Media (LiveKit & AI)	LIVEKIT_EXTERNAL_URL	Adres WebSocket serwera SFU dla klientów.	**wss://livekit.voicemessenger...**
	LIVEKIT_API_KEY/SECRET	Pary kluczy do generowania tokenów dostępowych.	**devkey / **********
	AI_URL	Endpoint modelu LLM (Hugging Face) do moderacji.	**https://router.huggingface.co/...**
	AI_API_KEY	Klucz API do serwisu AI.	**hf_**********
Storage (MinIO)	MINIO_URL	Wewnętrzny adres API (używany przez backend).	**http://3.71.11.3:9000**
	MINIO_EXTERNAL_URL	Publiczny adres CDN (używany przez frontend).	**https://cdn.voicemessenger...**
Bazy Danych & Broker	MONGO_...	Konfiguracja połączeń do poszczególnych baz MongoDB.	**admin / **********
	POSTGRES_...	Dane uwierzytelniające do bazy autoryzacyjnej.	**postgres / **********
	RABBITMQ_...	Dane dostępowe do brokera wiadomości (AMQP).	**user / **********

7. Infrastruktura Wdrożeniowa i Sieciowa

W celu umożliwienia publicznego dostępu do aplikacji hostowanej w lokalnym środowisku kontenerowym zaprojektowano i wdrożono hybrydową infrastrukturę sieciową wykorzystującą serwer pośredniczący VPS. Poniżej opisano kluczowe komponenty tego rozwiązania.

7.1. Tunelowanie Ruchu (FRP - Fast Reverse Proxy)

Ze względu na brak publicznego adresu IPv4 w lokalizacji serwera aplikacji, wykorzystano narzędzie **FRP** do zestawienia bezpiecznego tunelu między środowiskiem lokalnym (Docker) a publicznym serwerem VPS (AWS).

Rozwiązanie to pozwala na:

- **Ekspozycję usług TCP/HTTP:** Ruch kierowany na publiczny port serwera VPS jest tunelowany do lokalnego kontenera API Gateway oraz serwisów WebSocket.
- **Obsługę protokołu UDP:** Jest to krytyczne dla WebRTC. FRP został skonfigurowany do przesyłania pakietów UDP w zakresach wymaganych przez serwery mediów (LiveKit oraz Coturn), co jest funkcją niedostępną w standardowych tunelach HTTP (np. Ngrok).
- **Obejście NAT (NAT Traversal):** Eliminacja konieczności konfiguracji przekierowania portów na routerze lokalnym dostawcy internetu.

7.2. Serwer HTTP i Automatyzacja SSL (Caddy)

Jako serwer brzegowy (Edge Server) na maszynie VPS zastosowano serwer **Caddy**. Pełni on rolę odwrotnego proxy (Reverse Proxy), przyjmując ruch przychodzący z Internetu przed przekazaniem go do tunelu FRP.

Kluczowe funkcje Caddy w projekcie:

- **Automatyczna terminacja SSL/TLS:** Caddy automatycznie zarządza certyfikatami bezpieczeństwa (Let's Encrypt), zapewniając szyfrowane połączenie HTTPS (dla API) oraz WSS (dla WebSocket) bez konieczności ręcznej rotacji certyfikatów.
- **Routing domenowy:** Kierowanie ruchu na podstawie nazwy domeny (SNI), co pozwala na obsługę wielu usług (np. API, panel Keycloak, MinIO) na jednym adresie IP.

7.3. Dynamiczny DNS (DDNS) i Zarządzanie Domeną

W projekcie wykorzystano usługę Dynamic DNS (w ramach domeny mywire.org), co pozwala na przypisanie stałej, przyjaznej dla użytkownika nazwy domeny do adresu IP serwera VPS.

Dzięki integracji **DDNS + Caddy + FRP**, architektura zapewnia:

1. **Stabilność adresu:** Użytkownicy końcowi korzystają ze stałej domeny (np. voicemessenger.mywire.org), niezależnie od ewentualnych zmian adresu IP serwera pośredniczącego.
2. **Przezroczystość:** Dla klienta końcowego (przeglądarki) cała infrastruktura tunelowania jest niewidoczna – aplikacja zachowuje się tak, jakby była hostowana bezpośrednio w chmurze publicznej.

8. Podsumowanie

Zrealizowany projekt systemu "Voice Messenger" spełnił postawione cele inżynierskie. Udało się stworzyć w pełni funkcjonalną platformę komunikacyjną, która łączy zalety architektury mikroserwisów z wydajnością hybrydowego modelu WebRTC. Szczególnym osiągnięciem jest opracowanie autorskiej infrastruktury wdrożeniowej opartej na tunelowaniu (FRP), która umożliwia bezpieczne wystawienie usług lokalnych do publicznego Internetu, co stanowi istotną wartość edukacyjną i praktyczną.